

```
[root@workbenchsvr dlsym_hook]# gcc -rdynamic -o dl_prog1 dl_
prog1.c -ldl
[root@workbenchsvr dlsym_hook]# ./dl_prog1
The value of x is 100
[root@workbenchsvr dlsym_hook]#
```

Now try to hook *dlsym()*, and you will get a segmentation fault, due to recursive calls (*dlsym()* will call itself). The following *dl_prog2.c* will be a recursive *dlsym()* call to itself, which causes a stack overflow and segmentation fault:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdint.h>
#include <dlfcn.h>

void *dlsym(void *handle, const char *name)
{
    void *(*dlsym_fn)(void *, const char *)=NULL;

    printf("inside shared object::before dlsym()...\n");

    dlsym_fn=dlsym(RTLD_NEXT, "dlsym"); /* this
    will call itself again and again */

    printf("inside shared object::after dlsym()...\n");

    return (*dlsym_fn)(handle, name);
}
```

Output:

```
[root@workbenchsvr dlsym_hook]# gcc -shared -ldl -fPIC dl_prog2.c
-o libdl_prog2.so

[root@workbenchsvr dlsym_hook]# LD_PRELOAD=/home/dibyendu/dlsym_
hook/libdl_prog2.so ./dl_prog1

inside shared object::before dlsym(...)

inside shared object::before dlsym(...)

Segmentation fault

[root@workbenchsvr dlsym_hook]#
```

Here is the code (*dl_prog3.c*) that successfully interposes *dlsym()*:

```
#define __USE_GNU
#include <stdio.h>
```

```
#include <stdlib.h>
#include <dlfcn.h>

extern void *__libc_dlsym (void *, const char *);
void *dlsym(void *handle, const char *symbol)
{
    printf("Ha Ha...dlsym() Hooked\n");
    void* result = __libc_dlsym(handle, symbol); /* now, this will
    call dlsym() library function */
    return result;
}
```

Output:

```
[root@workbenchsvr dlsym_hook]# gcc -shared -ldl -fPIC dl_prog3.c
-o libdl_prog3.so
[root@workbenchsvr dlsym_hook]# LD_PRELOAD=/home/dibyendu/dlsym_
hook/libdl_prog3.so ./dl_prog1
Ha Ha...dlsym() Hooked
The value of x is 100
[root@workbenchsvr dlsym_hook]#
```

What else can we do?

I have already interposed other functions like *getaddrinfo()*, *open()*, etc, so you can intercept any function you want. But there are a few limitations:

- Do beware of functions that themselves call *dlsym()*, when you need to call *__libc_dlsym(handle, symbol)* in the hook.
- Ensure that the SUID bit is not set, otherwise you can't use *LD_PRELOAD*.
- Also, internal library function calls are resolved before runtime—say, if some function in *libc* calls *getaddrinfo()* or *malloc()*, it will never call the hook from a different library. 

References

- Tutorial: Function Interposition in Linux by Jay Conrad http://www.jayconrad.com/cgi/view_post.py?23
- <http://en.wikipedia.org/wiki/Hooking>
- Linux man pages: *dlopen()*, *dlsym()*, *dlsym_r()*, *dlsym_r()*

By: Dibyendu Roy

The author is a Linux fundamentalist. He works as a Linux developer with a large organisation, and spends most of his time studying and exploring the uncharted spaces of C and Linux. He dreams of becoming a guitar player some day, and spends precious moments with LP, Cradle Of Filth and Children Of Bodom. Being an open source activist, he uses Linux for every silly little thing. He can be contacted at diby.roy@gmail.com.